

# Camera Fit (Rect + Perspective + Orthographic) Documentation

It is a versatile tool designed to enhance Unity projects by simplifying camera management. You can effortlessly align and fit camera to positions or meshes perfectly within the camera's view.

- [1. Frequently Asked Questions \(FAQs\)](#)
- [2. How To Use](#)
  - [Axis Selection](#)
  - [Focus Selection](#)
  - [Rect Selection](#)
  - [Shortcuts](#)
  - [Component](#)
- [3. Lerp Fit](#)
- [4. DoTween Support](#)
- [5. Examples](#)
- [6. Contact Information and Links](#)

If you have any questions, **please read through the FAQs** and try searching for the answers there. If you cannot find an answer there, please report the issue or write an email to [kako.tools.help@gmail.com](mailto:kako.tools.help@gmail.com).

If you report a bug, it really help me if you include any steps to reproduce it. Also, if you've got a feature you'd like to see implemented, please let me know.

# ***1. Frequently Asked Questions (FAQs)***

**Q. Is it easy for beginners to use Kako Camera Fit?**

**A.** Yes, there is nothing complicated. It does not require setup and you can use it immediately after importing.

**Q. I had some errors after import. Why?**

**A.** Firstly, try restarting Unity and check again. Secondly, try re-importing the asset. If none of these help, please contact me by email [kako.tools.help@gmail.com](mailto:kako.tools.help@gmail.com).

**Q. After importing I have pink models. What is happening?**

**A.** You probably added URP support when you didn't use it before. You can remove this renderer from the Quality and Graphics settings or change material shaders into built-in shaders.

**Q. I need to fit objects into a rotated rectangle. How can I do that?**

**A.** Rotated rectangle fitting is not currently supported. I plan to add this feature in upcoming updates.

**Q. I updated the asset's code, but now I'm encountering errors. What should I do?**

**A.** first ensure that you've followed the update instructions provided in the documentation or release notes. Double-check that all necessary dependencies are installed and that you've properly integrated the updated code into your project. If errors persist, please contact me by email [kako.tools.help@gmail.com](mailto:kako.tools.help@gmail.com).

## 2. How to Use

The usage of this tool is very simple. This tool includes shortcuts for the Camera object. You can execute the fit operation directly from a reference to Camera object. If you want space between objects and the border, you can give values to the padding parameters. It is automatically determined whether the fit operation should be horizontal or vertical. To frame desired 3D world-space points using camera extensions provided by Kako Camera Fit, you can follow examples below:

```
using Kako.CameraFit;

public class ExampleCameraFitter : MonoBehaviour
{
    public Camera cam;
    public Transform[] fitObjects;
    public float paddingX;
    public float paddingY;

    private void Start()
    {
        cam.Fit(GetPositions(), paddingX, paddingY);
    }

    private Vector3[] GetPositions() => fitObjects.Select(obj => obj.position).ToArray();
}
```

### ▪ Axis Selection

You can apply the methods shown below to perform the camera fit operation in a single axis. Other axis is not included in the calculation.

```
cam.HorizontalFit(GetPositions(), paddingX);
cam.VerticalFit(GetPositions(), paddingY);
```

### ▪ Focus Selection

You may want a certain point to be in the center during the fit process. If you give a value to the focus parameter, the camera will center itself on the given point and fit the other points to remain within the frame.

```
public Transform focusObject;

cam.Fit(GetPositions(), paddingX, paddingY, focusObject.position);
cam.HorizontalFit(GetPositions(), paddingX, focusObject.position);
cam.VerticalFit(GetPositions(), paddingY, focusObject.position);
```

## ▪ Rect Selection

If there are UI objects in certain parts of the screen and you do not want the game area to intersect with these objects, you can create a rect transform on the Canvas and then perform the fit operation using this rect transform. The camera will now use the given rect transform frame instead of its own frame. You must also specify the canvas where the rect transform is located.

```
public Canvas mainCanvas;
public RectTransform fitArea;

// Without Focus
cam.Fit(GetPositions(), paddingX, paddingY, fitArea, mainCanvas);
cam.HorizontalFit(GetPositions(), paddingX, fitArea, mainCanvas);
cam.VerticalFit(GetPositions(), paddingY, fitArea, mainCanvas);

// With Focus
cam.Fit(GetPositions(), paddingX, paddingY, fitArea, mainCanvas, focusObject.position);
cam.HorizontalFit(GetPositions(), paddingX, fitArea, mainCanvas, focusObject.position);
cam.VerticalFit(GetPositions(), paddingY, fitArea, mainCanvas, focusObject.position);
```

## ▪ Shortcuts

Below you can find the shortcuts you can use to fit the camera.

*Fit(Vector3[] pointList, float paddingX, float paddingY, Vector3? focus = null)*

*HorizontalFit(Vector3[] pointList, float paddingX, Vector3? focus = null)*

*VerticalFit(Vector3[] pointList, float paddingY, Vector3? focus = null)*

*Fit(Vector3[] pointList, float paddingX, float paddingY, RectTransform fitArea, Canvas canvas, Vector3? focus = null)*

*HorizontalFit(Vector3[] pointList, float paddingX, RectTransform fitArea, Canvas canvas, Vector3? focus = null)*

*VerticalFit(Vector3[] pointList, float paddingY, RectTransform fitArea, Canvas canvas, Vector3? focus = null)*

**pointList:** List of 3D world-space points that you want the camera to fit. The camera will adjust its position and size to encompass these points.

**paddingX:** Horizontal padding applied. It determines the additional empty space, in world units, to leave on the left and right sides of the framed area.

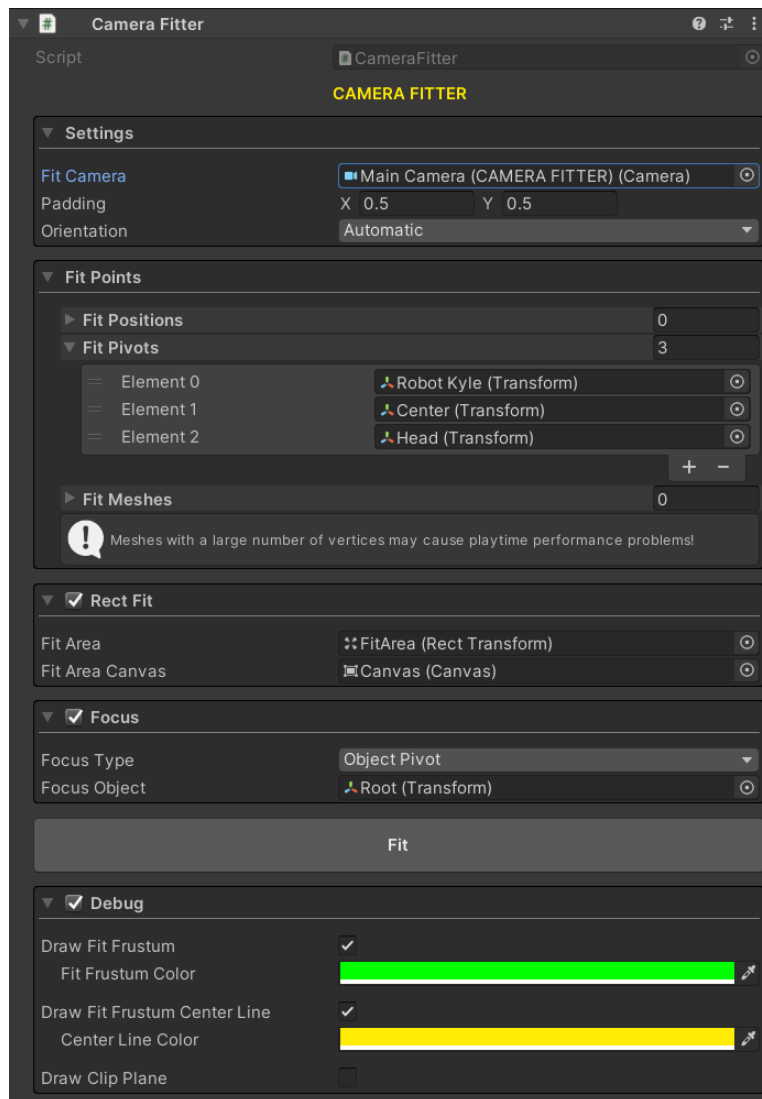
**paddingY:** Vertical padding applied. It determines the additional empty space, in world units, to leave on the up and down sides of the framed area.

**fitArea:** The camera fitting operation ensures that the specified pointList are visible within this RectTransform.

**canvas:** The Canvas component to which the fitArea belongs.

**focus:** If PROVIDED, centers the camera to specific point (in world space). If NOT PROVIDED, centers the camera to center of pointList.

## ▪ Component



The camera fitter component is accessible through the Component menu under

**KAKO ► Camera Fitter.** This component is designed to enhance visual and operational convenience, allowing for fitting operations directly within the editor and enabling debug functionalities.

```
public CameraFitter fitter;  
  
private void Start()  
{  
    fitter.Fit();  
}
```

### 3. Lerp Fit

Linearly interpolates the camera to fit area by the interpolant  $t$ . The parameter  $t$  is clamped to the range  $[0, 1]$ . This is most commonly used to gradually fit the camera.

When  $t = 0$ , keeps camera remains in its initial state.

When  $t = 1$ , ensures that the camera is on target fit state.

When  $t = 0.5$ , ensures that the camera is on midway between initial state and target fit state.

```
public Camera cam;
public Transform[] fitObjects;
public float paddingX;
public float paddingY;
public float fitSpeed;

private void LateUpdate()
{
    cam.Fit(GetPositions(), paddingX, paddingY).Lerp(Time.deltaTime * fitSpeed);
}

private Vector3[] GetPositions() => fitObjects.Select(obj => obj.position).ToArray();
```

or

```
public CameraFitter fitter;
public float fitSpeed;

private void LateUpdate()
{
    fitter.Fit().Lerp(Time.deltaTime * _lerpSpeed);
}
```

#### **Lerp(float t)**

Adjust the camera's position and size smoothly to fit.

**fitAction:** Desired camera fit action that is offered by Kako Camera Fit.

**t:** Value used to interpolate between initial fit and final fit.

## 4. DoTween Support

In order to add DoTween support, please go to **Assets ► KAKO ► CameraFit**. Then import TweenExtensions unity package by opening it. Then you can use it as shown in the following example.

```
public Camera cam;
public Transform[] fitObjects;
public float paddingX;
public float paddingY;
public float animDuration;
public Ease ease = Ease.OutQuad;

private void Start()
{
    cam.Fit(GetPositions(), paddingX, paddingY).DOFit(animDuration, ease);
}

private Vector3[] GetPositions() => fitObjects.Select(obj => obj.position).ToArray();
```

or

```
public CameraFitter fitter;
public float animDuration;
public Ease ease = Ease.OutQuad;

private void Start()
{
    fitter.Fit().DOFit(animDuration, ease);
}
```

**DOFit(float duration, Ease ease = Ease.OutQuad)**

Adjust the camera's position and size smoothly to fit.

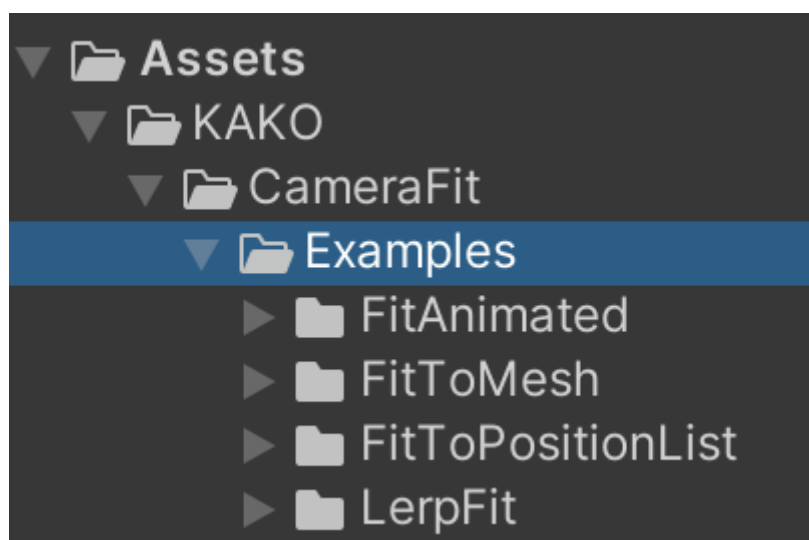
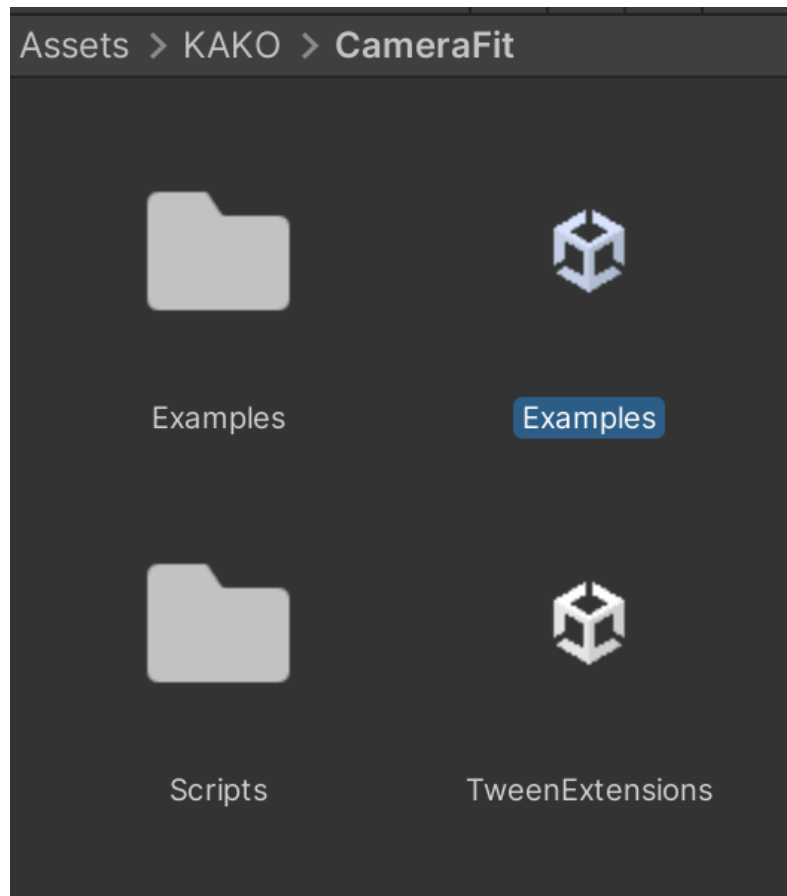
**fitAction:** Desired camera fit action that is offered by Kako Camera Fit.

**duration:** The duration of the sequence.

**ease:** If APPLIED to Sequences eases the whole sequence animation.

## 5. Examples

In order to add example scenes, please go to **Assets ► KAKO ► CameraFit**. Then import Examples unity package by opening it. Then you can view all of the examples.





## **6. Contact Information and Links**

- [Camera Fit \(Rect + Perspective + Orthographic\) at the Asset Store](#)
- Email: [kako.tools.help@gmail.com](mailto:kako.tools.help@gmail.com)
- Website: <https://sites.google.com/view/kako-tools>